

THE COMPLETE HANDS-ON GUIDE

Ollama

Talking to Your Models from Python

Now that Ollama runs on your machine, it's time to drive it with code. This phase teaches the official Python library from the ground up — sending prompts, reading the response object field by field, and managing your models entirely from Python.

PHASE 2 OF 6

Python Basics with Ollama

Phase 2 — Python Basics with Ollama

In Phase 1 you typed commands into a terminal. That's perfect for exploring, but the real power of a local LLM comes when your own programs can call it. The official `ollama` Python library makes this almost effortless — a few lines of code and your script is having a conversation with a model. By the end of this phase you'll send prompts two different ways, understand every field the model sends back, and control your whole model library from Python.

How the library relates to Phase 1. Remember the server on `localhost:11434`? The Python library is simply a polite, well-typed client that sends HTTP requests to that exact same server. Every function you'll learn here (`chat`, `generate`, `list` ...) maps directly onto a command you already met in Phase 1. Nothing new is running — you're just talking to it in Python.

WHAT YOU'LL LEARN IN THIS PHASE

Installing the library · single-turn prompts with `generate()` · message-based prompts with `chat()` · the exact structure of the response object · listing, inspecting, pulling and deleting models from Python · a full reference of every library function.

2.1 Installing the library — `pip install ollama`

2.2 `generate()` — single-turn prompts

2.3 `chat()` — the message-list format

2.4 Understanding the response object

2.5 `list()` — list models from Python

2.6 `show()` — inspect model metadata

2.7 `pull()` and `delete()` from Python

2.8 Handling errors gracefully

2.9 Full Python function reference

2.10 Hands-on project: AI Inventory & Ask CLI

2.1 — Installing the library

The library works with **Python 3.8 or newer** and is installed from PyPI like any other package. Ollama itself (the server from Phase 1) must already be installed and running.

```
# Install the official Ollama Python library
pip install ollama
```

It's good practice to work inside a virtual environment so your project's dependencies stay isolated:

```
# Create and activate a virtual environment (macOS / Linux)
python3 -m venv .venv
source .venv/bin/activate
pip install ollama
```

PREREQUISITE CHECKLIST

Before any Python code will work: (1) Ollama is installed, (2) the server is running (it usually starts automatically), and (3) you've pulled at least one model, e.g. `ollama pull gemma3`. If in doubt, run the health-check script you built in Phase 1.

Verify the install from a Python shell:

```
import ollama
print(ollama.list()) # should print your installed models
```

2.2 — `generate()`: single-turn prompts

`generate()` is the simplest way to get an answer: you hand it a model name and a single prompt string, and it returns one completion. There's no concept of a conversation here — each call is independent and remembers nothing about previous calls. This makes it ideal for one-off tasks: summarising a paragraph, classifying a sentence, generating a title.

```
import ollama

response = ollama.generate(
    model='gemma3',
    prompt='Explain what an API is in one sentence.',
)

# The generated text lives in the .response field
print(response.response)
An API is a set of rules that lets one program request services or data from another.
```

The single most important thing to remember: for `generate()`, the text you want is in `response.response` (or, dictionary-style, `response['response']`). We'll dissect the rest of the object in section 2.4.

Adding a system instruction and options

You can shape the model's behaviour with a `system` string and tune generation with `options` (we explore these fully in Phase 3 and 5):

```
response = ollama.generate(
    model='gemma3',
    prompt='Write a tagline for a coffee shop.',
    system='You are a witty marketing copywriter.',
    options={'temperature': 0.9}, # higher = more creative
)
print(response.response)
```

GENERATE() VS CHAT() — WHICH DO I USE?

Use `generate()` for stateless, one-shot text tasks. Use `chat()` (next section) whenever you want a back-and-forth conversation, roles, or to keep history. In modern Ollama, `chat()` is the more common choice — but `generate()` stays the cleanest tool for simple "input string → output string" jobs.

2.3 — `chat()`: the message-list format

`chat()` is built around the same **messages** structure used by most modern chat models. Instead of a single prompt string, you pass a *list of messages*, where each message is a dictionary with a `role` and `content`.

```
import ollama

response = ollama.chat(
    model='gemma3',
    messages=[
        {'role': 'user', 'content': 'Why is the sky blue?'},
    ],
)

# For chat(), the text is at response.message.content
print(response.message.content)
The sky appears blue because of a phenomenon called Rayleigh scattering...
```

Notice the difference from `generate()`: the answer is now nested inside a **message** object, at `response.message.content` (or `response['message']['content']`). The model's reply also carries a role of `'assistant'`.

The three roles you'll use are:

Role	Meaning
<code>system</code>	Background instructions that set the model's persona and rules (optional, usually first).
<code>user</code>	What the human says.
<code>assistant</code>	What the model has said previously — you include past replies here to give it memory.

A message list with all three roles looks like this. (Building real multi-turn conversations by appending to this list is the heart of Phase 3.)

```
messages = [
    {'role': 'system', 'content': 'You are a concise geography tutor.'},
    {'role': 'user', 'content': 'What is the capital of France?'},
    {'role': 'assistant', 'content': 'Paris.'},
    {'role': 'user', 'content': 'And its population?'},
]
response = ollama.chat(model='gemma3', messages=messages)
print(response.message.content)
```

DICTIONARY OR ATTRIBUTE — YOUR CHOICE

Every response supports both styles. `response.message.content` (attribute access) is cleaner and gives editor autocompletion; `response['message']['content']` (dictionary access) is handy when iterating or serialising. They return the same value.

2.4 — Understanding the response object

Both `generate()` and `chat()` return a rich object — not just the text. Knowing its fields lets you measure performance, detect why generation stopped, and debug problems. Here's an annotated `chat()` response:

```
# Inspect the whole object
print(response)

model='gemma3'
created_at='2026-05-30T11:20:05Z'
done=True
done_reason='stop'
message=Message(role='assistant', content='The sky appears blue...')
total_duration=4883583458      # nanoseconds for the whole call
load_duration=1019800         # time to load the model
prompt_eval_count=26          # tokens in your prompt
eval_count=298                # tokens the model generated
eval_duration=4799921000      # nanoseconds spent generating
```

The fields you'll reach for most often:

Field	What it tells you
<code>.model</code>	Which model produced the answer — useful when your code can switch models.
<code>.message.content</code> (chat) <code>.response</code> (generate)	The actual generated text. This is what you print or use.
<code>.done</code>	<code>True</code> when generation has finished. Important when streaming (Phase 4), where early chunks have <code>done=False</code> .
<code>.done_reason</code>	Why it stopped — usually <code>'stop'</code> (natural end) or <code>'length'</code> (hit the token limit).
<code>.eval_count</code>	Number of tokens generated — a good proxy for response length and cost.
<code>.total_duration</code>	Total time for the request, in nanoseconds. Divide by 1e9 for seconds.

A practical use: measuring speed (tokens per second)

```
tokens = response.eval_count
seconds = response.eval_duration / 1e9
print(f'Speed: {tokens / seconds:.1f} tokens/sec')
Speed: 62.1 tokens/sec
```

WHY ARE DURATIONS IN NANoseconds?

Ollama reports timing in nanoseconds for precision. To convert: `seconds = value / 1,000,000,000` (i.e. `value / 1e9`). This is the standard way to benchmark your hardware's speed on a given model.

2.5 — `list()`: listing models from Python

The Python equivalent of `ollama list`. It returns an object containing your installed models, which you can loop over programmatically.

```
import ollama

models = ollama.list()
for m in models.models:
    print(m.model, '-', round(m.size / 1e9, 2), 'GB')
gemma3:latest - 3.3 GB
llama3.2:1b - 1.3 GB
```

Each entry exposes fields like `.model` (name and tag), `.size` (bytes on disk), `.modified_at`, and a `.details` block with the family, parameter size and quantization level.

2.6 — `show()`: inspecting model metadata

When you want to know what's *inside* a model — its parameters, its built-in system prompt, its template, its license — use `show()`. This is the Python equivalent of `ollama show`.

```
info = ollama.show('gemma3')

print(info.details.family)           # e.g. 'gemma3'
print(info.details.parameter_size)   # e.g. '4.3B'
print(info.details.quantization_level) # e.g. 'Q4_K_M'
print(info.modelfile)                 # the full Modelfile (see Phase 5)
print(info.template)                 # the prompt template
```


A PEEK AHEAD AT PHASE 5

The `modelfile` and `template` fields shown here are exactly what you'll learn to write yourself in Phase 5, when you create custom models. `show()` is the best way to study how existing models are built.

2.7 — `pull()` and `delete()` from Python

You can manage your whole model library without ever leaving Python. `pull()` downloads a model; `delete()` removes it.

```
# Download a model (this can take a while for large models)
ollama.pull('llama3.2:1b')

# Delete a model from disk
ollama.delete('llama3.2:1b')
```

`pull()` can also **stream progress**, which is perfect for showing a download bar in your own tools:

```
for progress in ollama.pull('gemma3', stream=True):
    print(progress.get('status'), progress.get('completed', ''))
pulling manifest
pulling 2af3b81862c6 ...
verifying sha256 digest
success
```

DELETE() IS PERMANENT

Just like `ollama rm`, `ollama.delete()` removes the model files from disk immediately. Double-check the name (with `list()`) before calling it in a script.

2.8 — Handling errors gracefully

Real programs must handle the case where a model doesn't exist or the server isn't running. The library raises `ollama.ResponseError`, which carries an `.error` message and a `.status_code`. A common, friendly pattern is to catch a 404 ("model not found") and pull the model automatically:

```
import ollama

model = 'does-not-exist-yet'
try:
    ollama.chat(model, messages=[{'role': 'user', 'content': 'Hi'}])
except ollama.ResponseError as e:
    print('Error:', e.error)
    if e.status_code == 404:
        print('Model missing – pulling it now...')
        ollama.pull(model)
```

2.9 — Full Python function reference

Every function in the library, mapped to the CLI command it mirrors from Phase 1.

Python function	Does what	CLI equivalent
<code>ollama.generate(model, prompt)</code>	Single-turn completion	<code>ollama run m "..."</code>
<code>ollama.chat(model, messages)</code>	Multi-message conversation	<code>ollama run m</code>
<code>ollama.list()</code>	List installed models	<code>ollama list</code>
<code>ollama.show(model)</code>	Show model metadata	<code>ollama show m</code>
<code>ollama.ps()</code>	List models loaded in memory	<code>ollama ps</code>
<code>ollama.pull(model)</code>	Download a model	<code>ollama pull m</code>
<code>ollama.delete(model)</code>	Delete a model from disk	<code>ollama rm m</code>
<code>ollama.copy(src, dest)</code>	Duplicate a model under a new name	<code>ollama cp</code>
<code>ollama.create(model, from_=...)</code>	Build a custom model (Phase 5)	<code>ollama create</code>
<code>ollama.embed(model, input)</code>	Generate embedding vectors (Phase 6)	—
<code>ollama.push(model)</code>	Upload a model to share (Phase 5)	<code>ollama push</code>

THE CLIENT OBJECT

Calling `ollama.chat(...)` uses a default client pointing at `localhost:11434`. To target a different host or add headers, create your own: `from ollama import Client; client = Client(host='http://localhost:11434')` then call `client.chat(...)`. The async version, `AsyncClient`, is the subject of Phase 4.

AI Inventory & Ask — a Python command-line tool

This project ties together everything in Phase 2 into one useful script: it reports your model inventory (`list`), checks what's loaded (`ps`), inspects a model's metadata (`show`), and lets you fire a one-shot question at it (`generate`) while printing real performance stats from the response object. It deliberately stays single-turn — conversations arrive in Phase 3.

The complete script — `ai_inventory.py`

```
# ai_inventory.py – inspect and query your local Ollama models
import ollama

def show_inventory():
    """Print every installed model with its size."""
    print('\n=== Installed models ===')
    for m in ollama.list().models:
        gb = round(m.size / 1e9, 2)
        print(f' {m.model:<22} {gb} GB')

def show_loaded():
    """Print models currently held in memory."""
    print('\n=== Loaded in memory now ===')
    running = ollama.ps().models
    if not running:
        print(' (nothing loaded)')
    for m in running:
        print(f' {m.model}')

def inspect(model):
    """Print key metadata for one model."""
    info = ollama.show(model)
    d = info.details
    print(f'\n=== {model} ===')
    print(f' family: {d.family}')
    print(f' size: {d.parameter_size}')
    print(f' quant: {d.quantization_level}')

def ask(model, prompt):
    """Send one prompt and report the answer plus speed."""
    r = ollama.generate(model=model, prompt=prompt)
    print('\n--- Answer ---')
    print(r.response.strip())
    secs = r.eval_duration / 1e9
    print(f'\n[{r.eval_count} tokens in {secs:.1f}s '
          f'f'= {r.eval_count / secs:.1f} tok/s, stop: {r.done_reason}']')
```

```

if __name__ == '__main__':
    show_inventory()
    show_loaded()

    model = input('\nModel to use (e.g. gemma3): ').strip() or 'gemma3'

    # Pull automatically if the model is missing (error handling from 2.8)
    try:
        inspect(model)
    except ollama.ResponseError as e:
        if e.status_code == 404:
            print(f'Pulling {model}...')
            ollama.pull(model)
            inspect(model)
        else:
            raise

    question = input('\nAsk a question: ')
    ask(model, question)

```

Run it

```
python ai_inventory.py
```

EXTEND IT YOURSELF

Three small challenges to cement the concepts: (1) add a menu option to `delete()` a model; (2) loop so the user can ask several questions without restarting; (3) let the user type two model names and run the same prompt through both, printing which was faster.

What you achieved

You installed the library, sent prompts with both `generate()` and `chat()`, read real fields off the response object (text, token counts, timing, stop reason), and managed models entirely from Python — with graceful error handling. You now have a working Python interface to your local AI.

COMING NEXT — PHASE 3: PROMPTING & CONVERSATIONS

You'll master the system/user/assistant roles, write effective system prompts, build real multi-turn conversations by appending to the messages list, manage context length, and use few-shot, chain-of-thought and structured-JSON prompting techniques.